Clover

Scoped Mathematical Reasoning with Obligation-Directed Types

A proof-language proposal for the fourth iteration of the Brainstorm campaign

Prepared for Vladimir Reshetnikov
GPT-6 Astra Pro
6 September 2026

Central proposal. Extend the author-facing typing discipline with scoped properties, quantified evidence, and dependent proof obligations. Make common mathematical moves into typed proof transformations. Resolve their routine premises by a bounded, proof-producing engine; keep Lean's kernel conversion unchanged in the first implementation.

Concrete contribution. A specified grounding algorithm; a restricted executable source-to-proof-plan-to-Lean pipeline; a realizable affine test-basis theorem for behavioral synthesis; and explicit protocols for unfinished claims, meaning preservation, and scope-safe replay.

Evidence status. The Python companion was executed. The Lean reference and generated Lean examples were not compiled in this environment. The full Clover surface, editor integration, and proposed performance study remain designs.

Abstract

Mathematicians usually state the transformation that matters—differentiate an identity on an open set, transport an exact quotient, restrict a construction to an invariant domain—and leave its routine premises implicit. Formal proof languages must nevertheless account for those premises, preserve the identity of the objects involved, and distinguish operations whose familiar notation conceals different semantics. Clover proposes an author-facing extension of Lean's typing discipline that treats this omitted work as a typed, scoped obligation problem rather than as an invitation to unrestricted proof search.

The design starts from both revised round-three syntheses, not from a fresh catalogue of features. It retains their conservative foundations, then makes four interfaces more concrete: quantified evidence and mathematical proof transformations; unfinished elaboration represented by dependent obligation telescopes; finite, goal-triggered rule generation with exact failure states; and behavioral abstractions equipped with realizability and coverage data. A test-basis theorem extends the usual affine-length criterion to correlated observations and restricted input domains. Worked examples preserve the precise hypotheses of the autonomous-derivative and Frey-curve proofs, explain naturality in a finite tensor construction, and expose a quantifier-order trap in almost-everywhere reasoning.

A small companion actually parses a restricted language, generates and checks proof plans, and exports ordinary Lean source. Its 35 regression tests pass; a separate exact-arithmetic companion performs 1,576 checks. These results do not establish a verified Lean frontend or a productivity improvement. They do expose a concrete inference-cost issue and demonstrate the reduction obtained by a narrower rule profile. The article concludes with a serious comparison of elaborator-level refinements and possible kernel extensions, a staged implementation plan, and an evaluation designed to distinguish language benefits from improvements already obtainable through libraries and automation.

Contents

1 The proposal and its boundary

Clover is a language for *specified mathematical reasoning steps*. Its smallest useful unit is not a sentence accepted because an AI found some proof nearby. It is a step with a fixed meaning, a typed conclusion, explicit local scope, and a checked account of the premises that justify it.

The author should usually write the mathematical distinction that selects the argument, not every fact that follows routinely from that distinction. In particular, the source should retain that an identity holds on an open region, that a quotient is exact in the integers before it is embedded into a field, or that an equivalence is natural in its target. It should not have to repeat neighborhood construction, denominator casts, and componentwise congruence at every use. The implementation should discover or assemble those details, without weakening the statement or silently choosing a different mathematical structure.

The extension has two levels. At the author-facing level, an expression has an underlying Lean type, an interface of proved properties, and possibly an unfinished obligation telescope. At the kernel level, accepted expressions remain ordinary Lean terms with ordinary dependent types. Thus Clover extends what the elaborator *uses when checking a program or proof*; it does not initially extend the kernel's notion of definitional equality.

This distinction is substantive. A conventional elaborator asks whether a term inhabits a type and resolves implicit arguments. Clover additionally asks which mathematical requirements a selected operation imposes, which scoped facts discharge them, which theorem-backed adapters are needed, and what remains unfinished. Its user-visible types include behavioral contracts even when the corresponding Lean function retains a familiar unrefined carrier type.

1.1 Four design commitments

Preserve meaning before searching for evidence. Objects, quantifier order, structures, domains, measures, and requested conclusion strength must be determined independently of whether a convenient proof is found. An authorized construction is different: its witness may be chosen, but its specification is fixed.

Make the mathematical move explicit. A source step such as `differentiate on U` selects a proof transformation with a precise interface. A source step such as `search` delegates the route. These are not two spellings of the same operation.

Treat pending conditions as pending. An elaborator may compute a conditional proof while some guards remain open. That conditional proof must not enter the evidence index as an unconditional fact, and an unfinished intermediate assertion must not disappear merely because the final theorem has another proof.

Bound routine inference; expose escape routes. The default engine works in a finite source-derived term arena and a selected theorem profile. When it cannot close an obligation, the author may supply a lemma, choose a larger profile, introduce a missing construction, or enter ordinary Lean. Failure of this engine is not mathematical falsity.

1.2 What is new here, and what is inherited

The round-three campaign already established most of the underlying refinement calculus. Calling the same calculus Clover would not by itself advance the project. The proposed increment is therefore operational and interface-level:

Area	Inherited foundation	Clover's additional specification
Properties	Evidence about a fixed object	Quantified evidence schemes and their controlled specialization.
Proof work	A frontier of missing premises	Dependent obligation telescopes with explicit pending/closed transitions.
Inference	Finite ground closure	A source-derived arena, conclusion-triggered instantiation, and bounded premise-only parameters.
Behavior	Sound abstraction and realizable refutation	Affine test bases for the actual observation image, including correlations and guards.
Implementation	Small semantic cores	An executed restricted parser, proof-plan generator, independent checker, and Lean exporter.

These contributions have different evidence levels. The calculus and general algorithms are specified and justified mathematically in the article. The endomorphism slice is executed Python. Its ordinary-Lean rule reference is written but not freshly kernel-checked here. Nothing in this package should be described as a completed general-purpose Clover compiler.

2 Reading the third round as an implementation brief

Both revised syntheses were read at Brainstorm commit `bf3333905995`: *From Properties to Proof Obligations* and *Nine Refinements* [1, 2]. They agree much more than their different tables suggest. One organizes ten architectural commitments; the other scores a finer-grained set of commitments and collects a larger mutation catalogue. These are editorial views of related proposals, not independent measurements of language quality.

Their strongest recommendation is to implement a small interface rather than produce another overlapping design. Clover accepts that recommendation in a limited but concrete way: the companion supplies an executable operational slice, while the article specifies the parts that must be added to make it a real Lean extension. It does not present the small model as having completed the exact-quotient or analytic integration tasks requested by the syntheses.

2.1 The inherited distinctions must survive compression

The reports separate *object*, *refinement*, *observation*, *construction*, and *operational receipt*. An object is a fixed term with its selected structures. A refinement is more evidence about that term. An observation is related information, such as a prefix, enclosure, or quotient image. A construction produces a potentially different witness. A receipt records what an implementation did; it is not automatically a proof of its mathematical claim. Clover preserves all five distinctions.

Several earlier mistakes show why they matter. A finite observation can support a universal proof through a sound theorem without making the method complete. A failing abstract length is not a source-level counterexample until it is realized by an admissible input. Normalizing a counterexample creates a new witness rather than another property of the old witness. A finite index set does not make its algebra factors finite as modules. A final contradiction can prove an arithmetic helper while bypassing the arithmetic argument that a benchmark intended to exercise. The revised syntheses explicitly correct these confusions [1, 2].

2.2 What Clover takes from the individual proposals

The following attribution follows the syntheses' detailed discussion; it is not a claim of a new audit of every original proposal and companion.

Proposal	Design lesson retained in Clover
Basalt	Properties of diagrams and coupled objects must remain relational; mathematical structures and null-boundary conditions are part of the proposition.
Fiber	Exact quotients should be tested under satisfiable, operation-specific assumptions; synthesis holes need contracts on the actual intermediate values.
Gneiss	Infinite objects may admit lawful finite observations only because an operation preserves the relevant locality.
Karst	Evidence is scope-sensitive; proposition-valued finiteness is not a chosen enumeration; `List Empty` tests negative realizability.
Moraine	A restricted syntax can support a universal behavior theorem through an interpreter and an affine model; naturality is not a fieldwise annotation.
Obsidian	Normalization changes witnesses; existing Lean facilities are serious controls; an inconsistent ambient package is a poor arithmetic benchmark.
Schist	Backward requirements express the way a selected mathematical method asks for premises; small complete checking chains are valuable.
Tephra	Transfer, coverage, and realization are different theorems; arbitrary polymorphic Lean definitions do not receive parametricity for free.
Trellis	Closure is predictable only after the finite inference fragment is fixed; bound quality and sharpness are different contracts.

2.3 Three gaps are more urgent than new notation

First, a finite closure theorem leaves rule generation unspecified. A practical system needs to say which theorem parameters it instantiates, where their values come from, and when enumeration stops. Section 9 answers that question for a conservative fragment and implements it for a smaller one.

Second, a fixed-object evidence row is incomplete as an authoring model unless it retains *quantified scope*. The autonomous-derivative proof works with an induction hypothesis valid throughout a region, not merely at its current point. Section 7 treats this as an interface requirement.

Third, source fidelity needs an actual protocol. A provider must not solve a statement parameter while claiming to solve a proof hole. A stale local name must not identify an object after a declaration has been rebuilt. Sections 8 and 10 specify these boundaries separately from theorem truth.

3 What the inspected Lean code actually teaches

The source studies below are development examples, not held-out benchmarks. They were inspected through immutable GitHub references; the repositories were not built or executed in preparing this article. The machine-readable source manifest records the exact paths and scope of review.

3.1 An autonomous differential equation: locality and quantifiers

In ProveIt's `AutonomousIteratedDeriv.lean`, the theorem `iteratedDeriv_eq_comp` proves a general mechanism behind higher derivatives [3]. Let $U \subseteq \mathbb{R}$ be open, let $W, \varphi : \mathbb{R} \to \mathbb{R}$, and let $G_n, G_n' : \mathbb{R} \to \mathbb{R}$. The hypotheses assert, only at the points actually needed, that

$$W'(x) = \varphi(W(x)) \qquad (x \in U), \tag{1}$$

$$G_0(W(x)) = W(x) \qquad (x \in U), \tag{2}$$

$$\frac{d}{dw} G_n(w)\Big|_{w=W(x)} = G_n'(W(x)) \qquad (x \in U), \tag{3}$$

$$G_{n+1}(W(x)) = G_n'(W(x))\varphi(W(x)) \qquad (x \in U). \tag{4}$$

The derivative hypotheses here mean *existence with the specified value*, as expressed by `HasDerivAt` in the source. They are stronger than a bare equality involving Lean's totalized derivative function.

The conclusion is

$$\forall n \in \mathbb{N}\ \forall x \in U, \qquad D^n W(x) = G_n(W(x)).$$

Its successor step first turns the regional induction hypothesis into equality in a neighborhood of x, then differentiates there and applies the chain rule. In the source, the key bridge is the application of `Filter.eventuallyEq_of_mem` to openness, membership, and the *unspecialized* induction hypothesis.

Two details are not incidental verbosity. The induction hypothesis must remain quantified over the region, and the derivative assumption for G_n is needed only at $W(x)$ for $x \in U$. Replacing the latter by global differentiability would simplify the interface by strengthening the theorem, not by explaining the original theorem more naturally.

The existing proof is already well factored. Once this general theorem exists, its applications may be shorter than any new narrative syntax. Clover's target is the repeated assembly of locality and scope information in proofs of this kind, not a claim that this particular theorem has no concise Lean formulation.

3.2 Frey coefficients: one notation, two divisions

The FLT source `S_FreyPackage_freyCurveInt_map.lean` proves that the integral curve model maps to the rational one [4]. Its nontrivial coefficient obligations compare

$$\left(\frac{b^p - 1 - a^p}{4} : \mathbb{Z}\right) : \mathbb{Q} \quad \text{with} \quad \frac{b^p - 1 - a^p}{4} : \mathbb{Q},$$

and the analogous expression with numerator $-a^p b^p$ and denominator 16. The source uses `Rat.intCast_div`, followed by the divisibility proofs needed to justify the cast. The paper syntheses also exhibit the corresponding `Int.cast_div` interface at their Lean/Mathlib pin.

The repetitive work has a precise shape: extract divisibility from the modular assumptions, lift it through powers, normalize a residue modulo four, establish exactness, and transport the resulting balance equation. Clover can package that shape. It must not make integer division acquire field-division semantics because their printed symbols look alike.

The same file also contains a short terminal `solution` that invokes the helper. Comparing a short Clover call with the entire source file, including other lemmas about invariants and valuations, would manufacture a misleading compression ratio. The meaningful comparison is between equally equipped interfaces at the same proof boundary.

3.3 A finite tensor construction: properties of an entire family

The FLT theorem in `S_Algebra_exists_algHom_equiv_pi.lean` constructs a commutative A-algebra W representing a finite family of algebra maps [5]. Its assumptions include that every factor H_i is finite and free as an A-module. Its conclusion includes both those module properties for W and a family of equivalences

$$e_T : \mathrm{AlgHom}_A(W, T) \simeq \prod_i \mathrm{AlgHom}_A(H_i, T)$$

that is natural in T.

The code carries these data through induction on the finite index type, transport along equivalences, the empty family, and the tensor-product step. A property interface must preserve the dependency of H_i on its index and the naturality equation for the *whole family* e_T. Independent tags saying that the individual e_T are equivalences would omit part of the requested construction.

This example also tests the difference between evidence and data. The source assumes `Finite` for the index type, not an author-supplied executable enumeration. A mathematical existence proof may use a different interface from a program required to compute a concrete tensor presentation.

3.4 Leant: preserve the staging boundaries that already exist

The directly inspected Leant source is pinned at `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. Its `Verified` type has a nominal role and an explicit comment limiting its meaning to acceptance by a supplied callback [6]. The Length contract module describes provider laws as passive assertions with explicit argument roles. The adapter binds a checked candidate origin to a specific Length problem and explicitly declines to treat a model-level failure as authority for source-level behavioral refutation [7, 8].

Both syntheses report a newer local Leant snapshot, `823259f7e3c6`, whose behavioral path checks an exact proposition about an exact candidate using Lean, and separately checks its negation before reporting falsification. That is useful reported evidence; it is not the same revision as the one directly fetched here [1, 2].

Clover should add a proof-retaining adapter at these boundaries, not weaken Leant's existing distinctions between candidate identity, callback acceptance, model evidence, and source semantics. Section 11 gives the proposed contract.

4 A small mathematical surface, not unrestricted prose

The full notation in this section is proposed syntax. Only the deliberately smaller grammar in Section 15 has an executable parser in this package.

4.1 Four ordinary authoring actions

Clover distinguishes introducing data, assuming premises, establishing facts, and constructing witnesses. A compact example is:

```
fix f : A -> B
fix g : B -> A
assume inverse : left_inverse g f
know injective f by left_inverse
```

The first two lines introduce fixed objects. The third adds an explicit premise. The fourth requests a proof of a consequence; it is not another assumption. The proof transformation behind `by left_inverse` must produce a proof of injectivity for that same f.

A refined binder is a convenient abbreviation:

```
fix x : Real where x > 0
```

means an ordinary real variable together with a premise that it is positive. By contrast,

```
let y := expression where y > 0
```

creates an obligation to prove positivity of the specified expression. A language that interprets both uses of `where` as assumptions would be unsound as an authoring interface even if its final generated Lean declaration were type-correct under the extra assumption.

Witness construction is visibly different:

```
construct g : B -> A
  satisfying left_inverse g f
  using finite_inverse_construction
```

Here the author has authorized a choice of g, but not a change of f, A, B, or the left-inverse specification. A successful construction returns data and its proof. An unsuccessful attempt does not assert that no inverse exists.

4.2 Operations and laws have different domains

Clover uses `operation` for an interface with proof-implicit arguments:

```
operation exact_quotient(n d : Int)
  requires nonzero : d != 0
  requires exact   : d divides n
  returns q : Int
  ensures d*q = n
```

Its ordinary-Lean meaning is a function accepting n, d, the required proofs, and returning a quotient with a balance proof. A call generates those proof arguments. This is an explicit partial-domain interface over Lean's total integer-division primitive; it does not change the primitive itself.

A `law` instead states conditional behavior of a function that already exists:

```
law quotient_balance for integer_division
  given n d : Int
  requires d != 0 and d divides n
  ensures d * integer_division(n,d) = n
```

The carrier function remains defined at every pair of integers. Only the advertised theorem is conditional. The two interfaces should not be conflated by a single ambiguous arrow notation.

4.3 Mathematical methods are typed transformations

A method declaration associates a source operation with a theorem-backed transformation of a proof problem. Representative methods are:

```
differentiate identity on U
transport equation through ring_map phi
restrict action to invariant P
compare all finite observations
induct n keeping x in U general
```

These are not magic words. Each selects an interface with required premises and a prescribed relationship between input statements and output statements. For example, differentiating a regional identity must produce local equality at the point of differentiation; restricting an action requires invariance of the predicate for the *selected* action.

A `hint` may guide proof search without constraining the accepted route. A `by method` step requires the corresponding derivation node. A `search` step delegates the route completely within the declared policy. This makes the distinction visible without pretending that software can infer a human author's intended proof method from an arbitrary proof term.

4.4 A readable proof has a checked structural spine

The proposed autonomous-derivative proof can be written as follows. Named hypotheses retain the exact meanings of (1)–(4).

```
prove for n : Nat, on U:
  D^n W = G(n) compose W
by induction n keeping the point in U general:
  zero:
    use G_zero
  next n:
    differentiate IH on U
      using chain_rule from W_derivative and G_derivative(n)
    identify the result using G_step(n)
```

The mathematical spine is induction, local differentiation, and recurrence. The expanded view contains the exact neighborhood fact, chain-rule application, and rewrites. If the author removes openness, the system must not silently accept the same method because its final target happens to have another proof. It should identify the locality premise this method can no longer discharge.

Clover's prose view is generated from this structured proof document. Arbitrary comments may still be written, but a displayed formula presented as an asserted step must have its own evidence link. The final theorem's proof is not enough to certify every sentence that happens to appear beside it.

5 The author-facing type system

5.1 Carrier types, property interfaces, and evidence

Let Γ be an ordinary Lean local context, including selected structure arguments. Let E be an index into proof terms available in that context. The index is not a second logical context: every accepted entry must have a Lean type and proof in Γ.

A closed Clover judgment has the form

$$\Gamma; E \vdash t \Rightarrow A\ [\Phi],$$

where $t : A$ is a fixed term and Φ is a finite interface of propositions with retained proofs. A proposition in Φ may concern t and other objects; an injectivity fact concerns a function, while an inverse-pair fact concerns two functions. A property interface is therefore not restricted to unary labels such as *positive* or *continuous*.

Its interpretation is deliberately simple:

$$\Gamma \vdash t : A, \qquad \Gamma \vdash p_i : \phi_i \quad (\phi_i \in \Phi).$$

Adding a property does not rebind t to a new subtype. At an exported interface, the same information may be packaged as

$$\mathsf{Ref}(A, P) := \{x : A \mid P(x)\},$$

or as a named structure with data and proof fields. Lean already has these representations; Clover makes their use-site requirements systematic [9].

There are four elementary rules. An accepted proof introduces a property; a proved implication weakens or transforms an interface; properties about one fixed configuration may be conjoined; and a checked equality may transport a property. None of these rules lets a proposition become data merely because it has a convenient English name.

For example, a proof that $a \neq 0$ does not construct an inverse of a in a ring. In $\mathbb{Z}$, the element 2 is nonzero and regular but not a unit. In $\mathbb{Z}/4\mathbb{Z}$, the element represented by 2 is nonzero but not regular. Clover keeps the predicates *nonzero*, *regular*, and *unit* distinct, with conversions supplied only by theorems valid for the selected structure.

5.2 Checking rather than guessing a stronger type

Clover should not infer a mythical strongest refinement of an arbitrary term. There are infinitely many true properties of even a simple numeral, and no single finite interface captures all useful mathematics about it.

Instead, the system is bidirectional. In synthesis mode it obtains an underlying type and a small interface exposed by the term's constructor, provider specification, or known local facts. In checking mode an operation requests a particular property interface. The obligation engine tries to produce the missing proofs, using the rules allowed at that use site.

Thus an author may request

$$f : A \to B \quad [\mathsf{Injective}(f)],$$

without requiring every function-valued expression to carry a permanently materialized injectivity field. The semantic subsumption step is an explicit proof-producing operation in elaboration, not an extra case in kernel conversion.

This approach resembles refinement-type inference in its use of selected qualifiers and implication obligations, but its predicates need not be confined to a decidable SMT fragment. Outside a chosen automatic fragment, they remain ordinary proof obligations. Liquid Types and refinement-type inference provide relevant design precedents, not a ready-made decision procedure for arbitrary Lean mathematics [18, 19].

5.3 Pure behavioral contracts

For a fixed total function $f : A \to B$, define

$$\mathsf{Beh}(f; P, Q) := \forall x : A,\ P(x) \to Q(x, f(x)).$$

The postcondition is relational: it can refer to the actual input and output. A sorting contract is not merely that the output is sorted. It includes permutation of the input:

$$\forall xs, \quad \mathsf{Sorted}_{\leq}(f(xs)) \ \wedge \ \mathsf{Perm}(xs, f(xs)).$$

Stability, a comparison-cost bound, or preservation of a separate invariant are additional requirements, not consequences of the bare function type. A cost claim needs a specified operational cost model; mathematical extensional correctness does not imply it.

Variance is proof-directed. Suppose $P'(x) \Rightarrow P(x)$ and $Q(x, y) \Rightarrow Q'(x, y)$. A contract from P to Q yields a contract from P' to Q'. The precondition becomes more restrictive and the guarantee weaker. There is no scalar ranking in which every possible contract is simply "stronger" or "weaker" than every other one.

Composition preserves the intermediate value. If

$$\mathsf{Beh}(f; P, Q), \qquad \mathsf{Beh}(g; R, S),$$

then a useful composition rule requires

$$\forall x, y, \ P(x) \wedge Q(x, y) \Rightarrow R(y).$$

It yields, for example,

$$\forall x, \ P(x) \Rightarrow Q(x, f(x)) \wedge S(f(x), g(f(x))).$$

A final output relation $T(x, g(f(x)))$ requires one more proved implication. Replacing the actual $f(x)$ by an unrelated existential witness would lose the connection that justifies the second call.

Higher-order and relational properties use the same discipline. Monotonicity, Lipschitz continuity, naturality, equivariance, and commutation of two maps are propositions about multiple evaluations or whole families. They are not approximated by a collection of unrelated unary output predicates.

5.4 Refined domains are not guarded total functions

A function

$$f : \{x : A \mid P(x)\} \to B$$

need not extend to $A \to B$. There may be no suitable output outside the refined domain, or the desired extension may require a choice of default or a separate construction. A guarded law about an existing total function does not have this difficulty because the data already exist everywhere.

Likewise, a proof of $\exists y, Q(y)$ is not generally an executable procedure returning such a y. Lean distinguishes proposition-valued existence from computational data; classical choice can provide a mathematical selection, but that is a different construction policy [11]. Clover's `construct` node records whether its result is computational or invokes an admitted noncomputable selection.

5.5 Existing Lean mechanisms are part of the baseline

Lean already supports automatic parameters whose associated tactic scripts supply missing arguments. It also supports typeclass inference and ordinary bundled properties [12]. Consequently, the mere syntax `requires` is not evidence of a new capability. A first implementation could lower it to automatic proof arguments backed by a shared resolver.

The proposed contribution beyond that baseline is the coordinated handling of quantified scope, a shared and inspectable obligation graph, role-checked method interfaces, controlled rule generation, and diagnostics that preserve the mathematical meaning of the missing premise. Whether that coordination justifies separate syntax is an empirical question.

6 Unfinished reasoning as a dependent obligation telescope

6.1 A pending result is a function awaiting evidence

An unfinished elaboration judgment has the form

$$\Gamma; E \vdash e \Rightarrow A[\Phi] \,!\, \Delta,$$

where

$$\Delta = (h_1 : P_1,\ h_2 : P_2(h_1), \ldots, h_m : P_m(h_1, \ldots, h_{m-1}))$$

is a well-formed telescope of proof obligations. In the core fragment, each binder in Δ is proposition-valued. Authorized data constructions are separate nodes that extend Γ with a chosen witness and the evidence that accompanies it.

The meaning is that a skeleton $e : A$ and the advertised property proofs exist under Γ, Δ. Before the obligations are discharged, the exported object is at most the abstraction

$$\lambda h_1 \ldots h_m.\, e : \Pi\Delta, A,$$

not an unconditional term of A. This is the precise sense in which unfinished proof work behaves like an *obligation effect*. It is not a new runtime effect and does not let the program read or manufacture evidence.

A pending expression may itself require proof arguments to be well typed. Its placeholder is therefore a suspended elaboration object, not an ordinary closed anchor that may already be used in arbitrary accepted facts. This restriction prevents a superficially convenient implementation from placing properties of ill-typed or incompletely specified data into the evidence index.

6.2 Sequential composition is dependent substitution

Suppose the first step produces a skeleton $y = e(x)$ and obligations Δ. The next method asks for a property $R(y)$ and produces obligations $\Theta(y)$. Composition forms the telescope

$$\Delta\ ;\ \Theta(e(x)),$$

with the actual intermediate substituted into every dependent requirement. If a proof in Δ is solved, it is substituted into later obligations; it is not simply deleted from a flat set of strings.

For exact quotient transport, the sequence is naturally ordered. First fix the integer quotient expression. Then prove its balance equation using the divisibility hypothesis. Map that equation to the target ring. Finally, if the requested conclusion contains division in that ring, supply a unit or field-nonzero premise for the mapped denominator. The final requirement is about the mapped denominator, not about an unrelated integer proposition with a similar name.

6.3 Discharging an obligation does not authorize a cycle

Assume a method gives a conditional theorem $P \to Q$. The implementation may retain this theorem while P remains open. It may not place Q among the unconditional facts and then use $Q \to P$ to solve the original guard. The two implications alone establish neither proposition.

Clover therefore distinguishes a *rule dependency graph*, which can have cycles, from an *accepted proof graph*, which is built from already available proofs and has no unsupported circular dependency. Induction and well-founded recursion are not forbidden: they are explicit logical rules with their own checked premises, not cycles in ordinary fact propagation.

A discharge operation must either supply a proof in the appropriate earlier context or retain the obligation. A branch-local proof is valid only under that branch's telescope. Leaving the branch exports a conditional theorem or a proof formed by case elimination; it does not export the local assumption as a fact in the parent scope.

6.4 The core translation theorem

Theorem 6.1 (Conditional preservation for the specified core)**.** *Assume that each registered method is accompanied by an ordinary Lean theorem of its declared type, that substitutions are well typed, and that object and structure arguments have been fixed as specified. A derivation of*

$$\Gamma; E \vdash e \Rightarrow A[\Phi] \,!\, \Delta$$

using the core introduction, application, refinement, transport, scope, and obligation-composition rules translates to an ordinary Lean skeleton of type A in Γ, Δ, with proofs of the propositions in Φ there. A well-typed closing substitution for Δ yields an ordinary Lean term of the advertised type in Γ.

Proof. Proceed by induction on the derivation. Variable and premise cases use the corresponding declarations in the context. A refinement-introduction case pairs the unchanged term with an available proof at the elaboration level; packaging at an interface uses subtype or structure introduction. Consequence applies the retained implication theorem. A method application instantiates its Lean theorem and applies it to the translated premises, introducing a proof binder for every still-open premise. Sequential composition uses the ordinary substitution lemma for dependent function types. Equality transport uses equality elimination with the actual predicate and indices. Branch entry extends the context; branch exit abstracts its assumptions or applies the specified case-elimination theorem. These operations produce terms in the claimed telescope. Applying a closing substitution to the resulting terms preserves their types and gives the final claim. □

This is a mathematical theorem about the stated rules. It is not a machine verification of the proposed parser, rule metadata validator, renderer, or scheduler. In particular, the theorem assumes the very object correspondence that a production implementation must establish. The more informative implementation obligations are the correspondence and state-transition tests in Sections 8, 10, and 15.

6.5 An obligation frontier that explains the next decision

A frontier entry should contain the exact proposition, its local context, its source span, the operation consuming it, and available sufficient routes. For example:

```
Step: transport q = n/d from Int to K
Open requirement: IsUnit (phi d)
Already available: d divides n; d != 0 in Int
Reason: this route rewrites the mapped balance as division in K
Alternative: keep the balance equation (phi d)*(phi q) = phi n
```

The last line describes an alternative *result*, not a silently weakened answer. Switching to that result changes the request and requires an author choice. Likewise, a premise listed by one route is not announced as necessary for every proof of the original theorem.

The first implementation need not find the smallest frontier or the weakest sufficient assumptions. Those optimization problems can be more expensive than finding a proof. It should report one or a few traceable routes and keep the original target fixed.

7 Quantified evidence and mathematical scope

7.1 Store a theorem scheme, not a cloud of instances

An evidence index must retain facts such as

$$h : \forall n \in \mathbb{N}\ \forall x \in U,\ P(n, x)$$

as quantified schemes. Storing only the instances already used loses the information needed for later reasoning under a new arbitrary point or index. Conversely, eagerly enumerating all instances is impossible even for the simplest infinite domain.

The index stores the binder telescope and its dependencies. A use site selects an instance by matching the requested proposition, supplying data arguments from the current scope, and proving its guards. The universal proof remains available unchanged. This is ordinary universal elimination with an operational index, not a new axiom about schematic formulas.

For the autonomous derivative proof, the induction step begins with

$$\mathrm{IH} : \forall y \in U,\ D^n W(y) = G_n(W(y)).$$

At an arbitrary $x \in U$, openness gives a neighborhood contained in U. Specializing IH *throughout that neighborhood* supplies the eventual equality needed for differentiation. Specializing IH to x first and forgetting its quantified form cannot justify the same step.

7.2 Region syntax is a binder discipline

The proposed block

```
on U:
  know f = g
```

means a checked universally quantified regional equality, not a rebinding of f and g to functions on a different domain. A local block

```
at x in U:
  know f(x) = g(x)
```

is weaker unless the surrounding proof establishes that x is arbitrary and exports the appropriate quantifier. The document model records that distinction before the pretty printer renders either block.

The same principle applies to dependent families. In a proof of $\forall n, \forall x \in U_n, P(n, x)$, the region depends on the index. An induction method must state what it generalizes and what it fixes; it cannot borrow an induction hypothesis over U_n as if it covered U_{n+1} without an inclusion or transport theorem.

7.3 Quantifier order is not decorative

A particularly sharp example comes from measure theory. On $[0, 1]$ with Lebesgue measure, define $f_t(x) = \mathbf{1}_{\{t\}}(x)$ and $g_t(x) = 0$. For every fixed t, the two functions are equal almost

everywhere. But there is no $x \in [0,1]$ at which they are equal for every t, because $f_x(x) = 1$. Thus

$$\forall t, \quad f_t =_{\text{a.e.}} g_t \quad \not\Longrightarrow \quad \text{for almost every } x,\ \forall t,\ f_t(x) = g_t(x).$$

For a countable parameter family, the implication does hold: intersect the countably many full-measure sets. This gives a precise contract for a `collect almost_everywhere` method: it needs countability or another proved uniform-null-set condition.

Clover should represent *for each parameter* and *almost everywhere* as different binders, not as unordered tags in a property row. This example also shows why the generic term "local" is too coarse. Neighborhood equality, regional equality, and almost-everywhere equality have different consumers and different quantifier laws.

7.4 Almost-everywhere consumers are relation-specific

For a fixed measure μ, an a.e. equality $f =_{\mu\text{-a.e.}} g$ supports an integral congruence through the appropriate theorem for the chosen integral. It does not support evaluation at a chosen point. If the interface also claims integrability or a finite integral, those properties must be carried through the corresponding integrability theorems; totalized notation alone does not supply them.

A consumer registration records the measure argument and the relation being used. It should distinguish a theorem that respects a.e. equality from a stronger theorem that extracts pointwise equality under continuity and support hypotheses. The latter is a separate mathematical result, not a coercion that can be inserted whenever point evaluation would be convenient.

7.5 Strategic steps: symmetry and without loss of generality

A language aimed at mathematical reasoning must support more than local fact inference. The phrase "without loss of generality" is a particularly useful test because it deliberately changes the configuration while preserving the original proof obligation.

Let X be a configuration type, $P : X \to \mathsf{Prop}$ its admissibility condition, $Q : X \to \mathsf{Prop}$ the goal, and $N : X \to \mathsf{Prop}$ a preferred normal position. A normalization method supplies $r : X \to X$ and proofs of

$$\begin{aligned} &P(x) \Rightarrow P(r(x)),\\ &P(x) \Rightarrow N(r(x)),\\ &P(x) \Rightarrow (Q(r(x)) \Rightarrow Q(x)). \end{aligned}$$

A proof of $\forall y,\ P(y) \wedge N(y) \Rightarrow Q(y)$ then proves the original $\forall x,\ P(x) \Rightarrow Q(x)$: apply it to $r(x)$ and use the last implication. These are the exact premises of the method, not a license to replace x by an equal object.

For a symmetric statement about two elements of a linear order, r can keep (x, y) when $x \le y$ and swap it otherwise. The method needs preservation of the hypotheses under swapping and the implication transporting the goal back. A proposed Clover step is:

```
without_loss x <= y
  using swap(x,y)
  preserving assumptions
  transporting goal by symmetry
```

The continuation works on the selected normalized configuration, while the outer declaration still proves the original statement. If the assumptions are not symmetric, the method must report that missing preservation proof; it may not copy them unchanged to the swapped variables.

This rule also explains the interaction between meaning preservation and legitimate data choice. The original target is sealed, but an explicitly selected proof transformation may construct new auxiliary data and prove a reduction back to that target. Freezing meaning is not a prohibition on normal mathematical argument; it is a prohibition on changing the claim without such a reduction. The general strategic rule is specified here, not implemented by the small endomorphism parser.

8 Fixing meaning without freezing legitimate constructions

8.1 Three kinds of holes

Clover classifies unfinished information by its semantic role rather than by whether it happens to be represented internally as a metavariable.

Role	Examples	Permitted resolution
Meaning-bearing	Carrier type, topology, measure, coefficient ring, branch, proposition parameter	Ordinary statement elaboration; unresolved ambiguity is reported or becomes an explicit rigid parameter.
Authorized data	A requested inverse, basis, factorization, algorithm, or representing object	A construction provider may choose a witness satisfying the frozen specification.
Proof	Nonzeroness, divisibility, continuity, a contract proof	Any admitted proof-producing provider may discharge the exact obligation.

The distinction is necessary because a proposition can be made easy by choosing its missing data. A solver asked to prove $x \geq 0$ must not set an unresolved meaning-bearing x to 0 unless the source requested construction of *some* nonnegative real. On the other hand, a request to construct a polynomial factorization may legitimately leave the factor coefficients as data holes.

8.2 The proposed sealing protocol

Statement elaboration first resolves notation and selected structures and produces a typed statement skeleton. The sealer then computes a dependency closure of the remaining metavariables: types of holes, universe constraints, instance arguments, and data occurring in the target all matter. Looking only for metavariables printed in the conclusion is insufficient.

Every remaining meaning-bearing component is either resolved before proof search, made an explicit rigid parameter of an authorized generalized request, or reported as an ambiguity. Making a parameter rigid must preserve the intended statement; it cannot silently generalize a source that asked for a particular object. The baseline implementation should prefer a diagnostic when that intent is not determined.

Proof and construction providers run in copied elaboration state. Their results are checked against the sealed target and the declared hole roles. The commit step examines assignments and rejects unauthorized changes to meaning-bearing variables or their dependency closure. A proof provider may create internal temporary metavariables, but every temporary needed by the accepted result must be solved or explicitly returned as an obligation.

Lean's elaboration APIs distinguish postponed and synthetic metavariables, but their internal categories should not be mistaken for this entire semantic policy. In particular, using an opaque proof metavariable does not by itself freeze every data or universe variable reachable from its type [13, 14].

8.3 Preserve the chosen structure, not just its carrier

A topology, order, multiplication, scalar action, or measurable structure is mathematical data. Two such structures on one carrier need not be equal. A proof that a fixed function is continuous under one topology does not apply under another topology merely because the function and carrier print the same.

A Clover lexical structure context records the actual elaborated dictionaries used in the block. The context is not an instruction to bypass Lean's instance system globally. It supplies explicit choices at selected interfaces and creates local bridges only where a library theorem expects an instance.

This policy may make generated instance-management preambles easier to understand, but no such reduction was measured here. The inspected Frey and tensor files are not evidence that every FLT file has a large avoidable preamble. Replacing one actual preamble is a separate experiment.

8.4 A precise but limited noninterference property

At the specification level, proof search should satisfy: if two accepted runs start from the same sealed statement and differ only in proof-provider choices, the resulting declarations have the same advertised proposition, up to the explicitly admitted equivalences of the sealing protocol. Different witnesses are allowed only in authorized construction positions.

The easiest first implementation enforces a stronger operational condition: no assignments to the protected metavariable closure, exact checking of the retained target, and no migration of local identities across rebuilt contexts. This can reject harmless changes, but it gives an implementable initial boundary. A later implementation may accept more cases through checked context or statement adapters; a matching hash alone is never such an adapter.

9 A finite inference engine with specified rule generation

9.1 What a registry entry contains

A registered rule is an ordinary theorem plus elaboration metadata. Its interface identifies the conclusion pattern, data parameters, premise parameters, structure parameters already fixed by the context, and permitted trigger positions. The metadata validator must compare those roles with the actual elaborated theorem type. A misspelled role annotation is an error, not permission to reinterpret the theorem.

The first automatic fragment uses Horn-shaped theorem interfaces:

$$\forall \vec{a}, \quad P_1(\vec{a}) \to \cdots \to P_k(\vec{a}) \to Q(\vec{a}).$$

All data in the instantiated propositions are fixed terms. The automatic fragment excludes general synthesis of new data from a proof premise, unrestricted higher-order unification, and dependent argument patterns whose meaning cannot be determined before their proofs are found. Those cases remain available through explicitly typed method applications and ordinary Lean elaboration; they are not promised to lie in the finite closure engine.

For example, the injectivity rule

$$\mathsf{LeftInverse}(g, f) \to \mathsf{Injective}(f)$$

has a data parameter g that occurs only in its premise. Searching for an inverse of f is not the same operation as instantiating an existing theorem with one of the functions already present. The finite engine does only the latter. A new inverse is an authorized construction request.

9.2 The term arena

For each supported data type, construct a finite arena from the elaborated terms occurring in the requested statement, visible assumptions, selected method arguments, and retained evidence relevant to the block. Include their well-typed subterms. Keep binder identity, universe instantiations, and structure arguments; do not identify terms because their pretty-printed strings agree.

The initial arena does not close under arbitrary function composition, polynomial formation, or constructor application. If f and g occur, that alone does not generate $g \circ f$, $f \circ g$, and all their iterates. A composition explicitly present in the target is already in the arena. A registered constructor may be used in a later, separately bounded widening phase or under an explicit `consider` request, but it must not create an unbounded hidden term universe during closure.

This restriction sacrifices some automatic proofs. It is also what makes the inference boundary intelligible: the engine is assembling consequences about a known set of mathematical objects, not searching simultaneously for every possible auxiliary object.

9.3 Goal-triggered grounding

The following procedure gives a complete finite grounding policy for the chosen arena and rule profile. “Match” below means a terminating, selected pattern-matching procedure, not arbitrary semantic unification.

```
work := [requested proposition]
seen := empty set of propositions
instances := empty set of ground theorem applications

while work is not empty:
  pop a proposition Q
  if Q is already in seen: continue
  add Q to seen
  for each enabled rule whose conclusion matches Q:
    bind the parameters fixed by that conclusion match
    enumerate remaining data parameters over their finite arenas
    retain only well-typed instances using arena terms
    record the exact theorem instance
    add its premise propositions to work

compute the forward closure of available proved facts
using these recorded instances
```

A candidate limit, term-size limit, or unsupported pattern can stop this procedure. Such a stop is a resource or fragment outcome. It must not be reported as completed finite non-derivability. In the reference implementation, exceeding the instance-attempt limit returns `resource_limit` and never returns a negative mathematical verdict.

A parameter that appears only in premises is enumerated from the arena. A parameter whose type depends on earlier data parameters is drawn only after those parameters have been instantiated and its type has been checked. If no finite arena for that dependent type is available, this rule application is outside the automatic fragment. Structure arguments are taken from the sealed

context rather than guessed from a global collection of alternative structures.

The production implementation should attempt exact evidence and cheap single-lemma matches before materializing the whole relevant closure. It can return immediately after a checked proof is found. Exhaustive grounding is needed only for the stated finite-fragment closure result, not as a condition for accepting every successful proof.

9.4 Closure, proof graphs, and a finite guarantee

Let F_0 be the available proved atoms and let $\mathcal{G}$ be the finite ground-rule set. Define

$$F_{n+1} = F_n \cup \{Q \mid (P_1, \ldots, P_k \Rightarrow Q) \in \mathcal{G},\ P_1, \ldots, P_k \in F_n\}.$$

Whenever a new atom is inserted, retain a proof node containing the exact rule instance and references to proofs already available for its premises.

Theorem 9.1 (Finite-fragment closure)**.** *For a fixed finite arena, a finite rule profile with terminating pattern matching, and completed grounding, the closure process terminates. Its final set is the least rule-closed superset of F_0. Any fair schedule reaches the same set of atoms. If the seed proofs and instantiated rule theorems are valid, every inserted atom has a valid proof. The selected proof, dependency support, and running cost need not be schedule-independent.*

Proof. Only finitely many instantiated atoms occur in F_0 and $\mathcal{G}$. Each strict change adds an atom, so there can be only finitely many changes. The limit is closed because every rule whose premises are present eventually fires. Every closed superset of F_0 contains each F_n, by induction, establishing leastness. A fair schedule cannot omit a consequence at the least fixed point, again by induction on the stage at which that consequence first appears. Each proof node applies an actual instantiated theorem to already justified premises, so induction on insertion order proves validity. Different schedules may choose different justifications for the same atom, which explains the last qualification. □

This is relative completeness for a *selected finite rule system*. It says nothing about all true propositions of Lean, all possible qualifier choices, or all auxiliary constructions a mathematician might invent.

With at most M arena terms per relevant type and at most v data parameters per rule, a simple upper bound on possible substitutions is rM^v for r rules, before accounting for dependent typing and pattern costs. This is a finite bound, not an attractive complexity guarantee. With a premise-indexed work queue and a residual-premise counter per ground rule, closure bookkeeping can be linear in the number of atoms and ground premise occurrences. Grounding, Lean type checking, normalization, proof retention, and rendering remain separate costs.

9.5 Profiles are interfaces, not global mathematical omniscience

A profile should name a small, coherent set of theorem interfaces: exact integer quotients, regional differentiation, finite affine lengths, or injectivity of compositions. It can import other profiles explicitly. Automatically enabling every theorem that resembles a property rule would recreate an unpredictable second elaborator.

The companion supplies a concrete warning. For one composition problem, the seven-rule registry generated 211 ground instances and retained a four-node proof. Restricting it to the two rules actually needed reduced that count to 16 while retaining a four-node proof. These are counts in the restricted Python model, not measurements of Lean inference. They nevertheless demonstrate

that "finite and demand-driven" does not imply "cheap": equality symmetry and transitivity can create a large relevant-looking search region even when the final proof never uses equality.

A sensible production policy is therefore staged: direct lookup; registered single-step rules; a small method-specific composition profile; explicit widening; finally broader external search. Each stage must retain its proof and outcome classification. No unsuccessful stage acquires authority to negate the target.

10 Transport, rewriting, and the lifetime of evidence

10.1 Three distinct ways to reuse a fact

Suppose an index contains $p : P(t)$ and a consumer asks for a fact about u. There are three common legal routes.

If t and u are definitionally equal in the fixed Lean context, ordinary type checking can reuse p. If a checked equality $e : t = u$ is available, equality elimination transports p to $P(u)$. If only a weaker relation $R(t, u)$ is known, reuse needs a consumer theorem of an appropriate shape, such as

$$R(t, u) \to P(t) \to Q(u).$$

The third route does not assert $t = u$ and must not populate the equality index as if it did.

The same applies when a simplifier rewrites propositions rather than their objects. From $p : P(t)$ and a proved equivalence $P(t) \leftrightarrow Q(u)$, the implication gives $Q(u)$ without proving $t = u$. The index may keep its original anchor and insert this bridge at the use site; it need not continually rewrite every stored fact into one global normal form.

10.2 Dependent indices make transport a typed operation

For $v : \mathsf{Vector}(A, n)$ and an equality $n = m$, a transported vector has a different indexed type. A property about the original vector is not reused by deleting its length index or matching a textual name. The adapter must carry the index equality, the transported value, and the predicate transport needed by the consumer.

Proof irrelevance can eliminate distinctions between proofs of a fixed proposition; it does not identify distinct data indices or structure arguments [10]. The implementation should initially support a few explicit transport shapes, keeping ordinary Lean as an escape hatch for more complicated dependent equalities.

A normalization policy must likewise distinguish an exact equality from a chosen isomorphism. Isomorphic vector spaces or representing algebras are not interchangeable data in every dependent expression. A library can supply coherent transport along an isomorphism, but the chosen map and its inverse remain part of that operation's meaning.

10.3 Snapshot-local indexing is the initial lifetime rule

Each evidence index belongs to an elaboration snapshot. Within it, keys use actual expressions and local identities. After an upstream edit rebuilds a declaration, the initial implementation reconstructs the index from the new Lean context rather than reusing old local identifiers.

More ambitious migration requires a typed context embedding. If

$$\Gamma = (x_1 : A_1, \ldots, x_n : A_n)$$

and the new context is Γ', a migration map must provide terms $\theta(x_i)$ whose types are the corresponding substitutions of A_i, in dependency order. Replaying a proof then means substituting θ into that proof and checking the result. A hash match or a matching source name is useful for locating a candidate migration, not for proving it correct.

This conservative rebuilding policy deliberately misses some reusable work. The restricted companion is even coarser: its snapshot key includes the entire source text, so even an unrelated edit invalidates old request identities. That is a safe prototype boundary, not the proposed final incremental editor architecture.

10.4 A framing law and its limits

Within a fixed signature and fixed data context, adding valid evidence cannot invalidate an already accepted proof. It can make new obligations solvable. For a fixed finite arena and rule profile, the closure operator is monotone in its seed facts. This is the evidence analogue of a framing property.

The qualification matters. Adding an import can change notation resolution or available instance choices during a *new* elaboration. Changing a method policy can make an old derivation unacceptable even when its theorem remains true. Changing a measure changes the proposition. These are not counterexamples to monotonicity because the sealed signature or acceptance policy has changed.

Clover should make that distinction visible in repair diagnostics: an old proof may fail because a fact disappeared, because the statement changed, or because the permitted method changed. A single undifferentiated "automation failed" message conceals the very information needed to repair the proof.

11 Behavioral synthesis and realizable observation types

11.1 The synthesis request is a dependent result type

For a requested program type T and fixed specification $S : T \to \mathsf{Prop}$, the accepted result is conceptually

$$\{f : T \mid S(f)\}.$$

A producer may search by types, examples, symbolic constraints, or language-model suggestions. The acceptance interface does not care which heuristic found f; it requires evidence of the *same* $S(f)$ for the *same* candidate.

A proposed Leant adapter retains three linked objects: the selected source edit and its elaborated term; the sealed specification with its instantiated types, universes, and structures; and a Lean proof of that specification or a checked certificate bridge producing one. Candidate identity and theorem correspondence are not reconstructed from a provider name or a pretty-printed expression.

The existing nominal callback receipt remains useful for staging. It should not be repurposed as the new evidence type. The additional receipt must state which target was checked, which proof or certificate is retained, and which axiom and execution policy was used. Replaying a different textual variant requires a new check of the edited source.

11.2 Soundness, completeness, and refutation are different contracts

Let D be an admissible concrete input domain, let $o : D \to U$ be an observation, and let an abstract evaluator describe the candidate on observations. Four questions must be answered independently.

Positive soundness: does acceptance of the abstract certificate imply $S(f)$ for all admissible inputs? A sound but incomplete method is already useful here.

Decision completeness: does the method accept every true specification in its stated program grammar and input domain? This is a stronger converse, not a prerequisite for accepting sound proofs.

Negative transfer and realization: does a reported abstract failure come from an admissible concrete input, and would the source specification imply the abstract property at that input? Both are needed for refutation.

Observation coverage: why do the available observations determine the property being claimed? A theorem over all finite prefixes is not the same thing as having checked several prefixes.

Clover represents these as different proof-bearing interfaces. A provider with only positive soundness can prove accepted cases. It may not label its failure as a counterexample. A provider with a concrete source counterexample may refute that candidate without possessing a complete decision procedure for the whole grammar.

11.3 A more general finite test basis for affine observations

The usual unrestricted affine-length criterion compares coefficients. That criterion is too strong for some restricted domains and can produce unrealizable negative witnesses. The right object is the affine geometry of the *realizable observation image*.

The following elementary theorem provides a precise reusable interface. Its value here is not a claim of new linear algebra; it is the way its hypotheses can be packaged as behavioral type information for synthesis.

Definition 11.1 (Realizable affine test basis)**.** Let K be a field, D a concrete admissible domain, and $o : D \to K^m$ an observation map. A realizable affine test basis consists of a base point b, directions $v_1, \ldots, v_r \in K^m$, and concrete inputs $x_0, x_1, \ldots, x_r \in D$ such that

$$o(x_0) = b, \qquad o(x_i) = b + v_i \quad (1 \le i \le r),$$

together with the coverage property

$$\forall x \in D, \quad o(x) - b \in \operatorname{span}_K\{v_1, \ldots, v_r\}.$$

The directions need not be linearly independent.

Theorem 11.2 (Affine specification from a realizable test basis)**.** *Suppose o has a realizable affine test basis. For affine functions $p, q : K^m \to K$,*

$$\big(\forall x \in D,\ p(o(x)) = q(o(x))\big) \quad \Longleftrightarrow \quad \bigwedge_{i=0}^{r} p(o(x_i)) = q(o(x_i)).$$

If one of these finite equalities fails, the corresponding x_i is an admissible concrete counterexample to the universal equality.

Proof. Write $p(u) - q(u) = c + \ell(u)$ for a linear functional ℓ. Equality at x_0 gives $c + \ell(b) = 0$. Equality at x_i then implies

$$0 = c + \ell(b + v_i) = c + \ell(b) + \ell(v_i) = \ell(v_i).$$

For arbitrary $x \in D$, coverage expresses $o(x) - b$ as a linear combination of the directions, so $\ell(o(x) - b) = 0$. Hence

$$p(o(x)) - q(o(x)) = c + \ell(b) + \ell(o(x) - b) = 0.$$

The reverse implication specializes the universal claim to the supplied inputs. A failing equality is already a failure at a named member of D, giving the last assertion. □

The proof separates useful assumptions. Coverage alone, together with checked values at the formal points $b, b + v_i$, suffices for a positive affine identity on the image. Concrete realizers make those checks necessary for the universal source property and turn failures into source witnesses. Full surjectivity of o onto K^m is unnecessary. If D is empty, the universal statement is vacuously true and is handled separately; there is no required base-point realizer.

11.4 Correlated lists: three observations, only two directions

Take $D = \mathsf{List}(A) \times \mathsf{List}(A)$, with a chosen inhabitant $a \in A$, and observe

$$o(xs, ys) = (|xs +\!\!+ ys|,\ |xs|,\ |ys|) \in \mathbb{Q}^3.$$

The image lies in the plane $n = p + q$. A realizable affine test basis is

$$b = (0, 0, 0), \qquad v_1 = (1, 1, 0), \qquad v_2 = (1, 0, 1),$$

realized by $([], [])$, $([a], [])$, and $([], [a])$. Every observation is $|xs|v_1 + |ys|v_2$.

Consequently, two affine expressions in total length, left length, and right length agree for all such list pairs exactly when they agree at these three realizable points. Their coefficient vectors in three independent variables need not be equal: n and $p + q$ are the simplest example. An abstract vector such as $(1, 0, 0)$ violates the observation relation and cannot refute the law.

For a difference

$$d(n, p, q) = c + an + bp + eq,$$

the complete criterion is

$$c = 0, \qquad a + b = 0, \qquad a + e = 0.$$

The companion enumerates 625 small integer-coefficient forms, checks this criterion against the finite test basis, and reconstructs actual list-pair witnesses for every failed basis check. It also performs finite grid checks for accepted forms. The universal theorem is the proof above; the finite run is an implementation check, not its replacement.

11.5 Empty element types and guards change the basis

If A is empty, both input lists must be empty. The observation image is just $\{(0, 0, 0)\}$, so a basis has one base point and no directions. This validates some universal source equations whose unrestricted affine coefficient vectors differ. It also explains why an abstract positive-length failure cannot be promoted to a source counterexample at `List Empty`.

A guard can have the same effect without an empty element type. On the domain of lists of length exactly 100, the length observation has image $\{100\}$. The affine expressions n and 100

agree on that domain although they are not identical affine functions on all natural lengths. A realizer requires an actual list of length 100; it must not be fabricated when the element type is empty.

More generally, a length guard or correlation defines a constrained observation image. Clover should ask for an adequacy package for that image rather than reuse an unrestricted coefficient criterion by default. There need not be a convenient computable basis for every guard. Failure to construct one leaves the method unsupported, not the specification false.

11.6 How this attaches to real candidate semantics

The test-basis theorem does not itself connect Leant's term graph to affine lengths. A complete adapter still needs an interpreter for the admitted candidate grammar, a normalization theorem computing its affine observation, and a correspondence theorem for the exact elaborated source candidate. Provider laws must be Lean theorems about the actual provider terms, not passive model assumptions promoted by name.

The sound path is therefore

$$\text{exact candidate} \longrightarrow \text{checked denotation bridge} \longrightarrow \text{affine model}$$
$$\longrightarrow \text{image-aware certificate} \longrightarrow \text{source contract proof.}$$

For a negative verdict, add the checked source input realizer and evaluate or prove the failure on that input. The companion's affine checker does not claim to implement this full path.

A polymorphic-looking type is not by itself a behavior theorem. Lean's reference gives a classical, noncomputable polymorphic list function that inspects its element type, invalidating an unrestricted parametricity rule [15]. A restricted synthesis grammar can support a separately proved parametricity theorem; membership in that grammar then becomes part of the evidence, not an inference from the printed function type.

Branching candidates may require piecewise affine models. Their adapter must prove coverage of the case domains and the denotation theorem in each case, respecting any case priority or overlap. A test basis for one branch cannot be used as coverage of the whole function. This is particularly relevant to Leant's explicit finite-spine case policy: enabling a structural case form is not itself a proof of a universal affine behavior law.

A failed completion refutes only that completion. Pruning an incomplete sketch requires a theorem that every admissible completion violates the requested contract, or a sound impossibility result for the entire remaining search space. One counterexample to one filled-in program is not such a theorem.

12 Two complete mathematical interfaces

12.1 Exact Frey quotients with separated premises

Set

$$N_2 = b^p - 1 - a^p, \qquad N_4 = -a^p b^p,$$

for $a, b \in \mathbb{Z}$ and $p \in \mathbb{N}$. The following proposition isolates the arithmetic used by the curve-base-change proof and slightly sharpens the sufficient lower bound needed by its first coefficient.

Proposition 12.1 (Separated exactness requirements)**.** *If* $2 \mid b$*,* $p \geq 2$*,* p *is odd, and* $a \equiv 3 \pmod 4$*, then* $4 \mid N_2$*. If* $2 \mid b$ *and* $p \geq 4$*, then* $16 \mid N_4$*, with no residue or oddness assumption on*

a or p. In each case, division by the indicated denominator in $\mathbb{Z}$ gives a quotient whose image in $\mathbb{Q}$ equals the corresponding rational quotient.

Proof. Write $b = 2c$. If $p \geq 2$, then $4 \mid b^p$. Since $a \equiv -1 \pmod 4$ and p is odd, $a^p \equiv -1 \pmod 4$. Therefore

$$N_2 \equiv 0 - 1 - (-1) = 0 \pmod 4.$$

If $p \geq 4$, then $16 \mid b^p$, and multiplication by $-a^p$ preserves that divisibility. For either numerator N and denominator d, exactness gives $N = dq$ for the integer quotient q. Mapping to $\mathbb{Q}$ yields $N = dq$ there; as $d \neq 0$ in $\mathbb{Q}$, division gives the desired rational identity. □

The lower bound for N_2 is a sufficient condition for this route, not a claim of a weakest possible contract. For particular a, b, p, exactness can hold under weaker assumptions. The operation's fundamental guard is simply the needed divisibility; the displayed arithmetic hypotheses are one registered route to it.

The proposed use site is:

```
fix a b : Int
fix p : Nat where p >= 4 and odd p
assume a == 3 modulo 4
assume 2 divides b

let q2 := exact_quotient(b^p - 1 - a^p, 4)
let q4 := exact_quotient(-a^p * b^p, 16)

know cast(q2, Rational) = (b^p - 1 - a^p)/4
  by transport_exact_quotient
know cast(q4, Rational) = (-a^p * b^p)/16
  by transport_exact_quotient
```

The rational expressions in the conclusions are elaborated over $\mathbb{Q}$ with the integer inputs explicitly embedded. Their meanings are fixed before the arithmetic profile runs. The expanded obligation graph records the first coefficient's residue and oddness route separately from the second coefficient's power-divisibility route.

The joint fixture $(a, b, p) = (3, 2, 5)$ gives $q_2 = -53$ and $q_4 = -486$. Removing one of the common sufficient premises has instructive neighboring cases: $(3, 2, 4)$ fails the first coefficient's exactness; $(3, 3, 5)$ fails it as well; $(3, 2, 3)$ fails the second coefficient's exactness; and $(1, 2, 5)$ fails the first. These are executed exact-arithmetic checks in the companion, while the general proposition above is a written proof.

A reusable helper should be exported with only its stated arithmetic premises, or checked in a deliberately restricted local context. Importing a full contradictory counterexample package would allow an irrelevant proof by contradiction. An assumption-support report helps detect such a bypass, but establishing satisfiability of the helper's premises with an explicit fixture is an additional and useful test.

12.2 What survives a general ring map

For a ring homomorphism $\phi : \mathbb{Z} \to R$, exactness always yields

$$\phi(N) = \phi(d)\phi(q).$$

Turning this balance into a division formula requires the target-side hypotheses of the selected operation. In a field, nonzeroness of $\phi(d)$ is enough. In a general ring, a chosen unit is the

relevant interface. Nonzeroness of d in the source is insufficient: its image may be zero in a positive-characteristic target.

This example motivates a useful Clover rule: a transport method exposes both what is preserved unconditionally and the additional premises needed by its requested consumer. It may propose the balance equation as an alternative, but it cannot silently deliver that weaker result when the source asked for a quotient identity.

12.3 A representing algebra as a dependent construction

For the finite tensor example, the natural Clover result is a dependent package:

```
construct W : CommutativeAlgebra A
  with finite_module W
  with free_module W
  with e : for each T : CommutativeAlgebra A,
             AlgHom(W,T) equivalent_to family i, AlgHom(H(i),T)
  with natural e in T
```

The declared input interface must retain the finite and free module properties of every H_i. For a finite enumerated family, an iterated tensor product supplies the object; without a chosen enumeration, a mathematical existence interface can follow the source's finite-type induction. These are different construction policies with related mathematical outputs.

The naturality field has an actual equation:

$$e_{T'}(u \circ f) = (i \mapsto u \circ e_T(f)_i)$$

for every A-algebra map $u : T \to T'$ and $f : W \to T$. The construction is not accepted merely because each individual e_T is bijective. Clover's obligation telescope preserves this universally quantified compatibility law.

The missing-factor negative case is especially valuable. With one factor, representability determines W up to an algebra isomorphism with that factor. Taking $A = \mathbb{Q}$ and $H = \mathbb{Q}[X]$ shows that finite indexing alone cannot force W to be a finite A-module: the monomials give arbitrarily large linearly independent finite families. A compact surface that dropped the factor's module hypothesis would be expressing a false theorem, not merely a more convenient version of the source theorem.

Finally, uniqueness up to isomorphism does not authorize substituting a different representing object into dependent data without transport. The selected W and selected equivalence family remain explicit construction results. Their proof fields may be implicit at use sites; their data identity may not.

13 Certified computation inside the same discipline

13.1 The CAS returns a relation to the original problem

Clover treats a computer-algebra operation as a producer of a value and evidence for a fixed relation. Useful result specifications include exact equality, equality under guards, equality on a region, a finite jet, a solution witness, a complete solution characterization, and an enclosure with an error bound. They are not points on a single ladder of confidence.

For example, an algorithm may produce y with $F(y) = 0$. That proves existence of a root, not completeness of a returned list of roots. A root-coverage theorem without a soundness theorem

for listed candidates is also insufficient for a complete solution set. A polynomial residual modulo X^N can identify a jet only with the additional hypotheses of the appropriate lifting theorem, including the chosen constant term and an invertibility condition where needed.

The same distinction governs familiar radicals. Over the reals, $\sqrt{x^2} = |x|$ holds for all x, while $\sqrt{x^2} = x$ requires $x \geq 0$. A symbolic simplifier that returns the latter expression must either supply the guard or preserve the absolute value. The author-facing property system makes such domain and branch information reusable, but does not turn a CAS convention into a theorem.

13.2 Checking and reification have different soundness obligations

A certificate checker can have an excellent theorem about an expression language while still be attached to the wrong original target. Clover therefore requires both a checker theorem and a reification bridge.

For a ring-expression certificate, the bridge fixes the coefficient ring, number of atoms, and valuation assigning each atom its exact Lean expression. It produces equalities between the original typed expressions and the denotations of their reified forms. The checker theorem proves the desired relation between those denotations. Composition of these proofs establishes the original request.

A basic ideal-membership certificate has the form

$$p = \sum_{i=1}^{s} q_i g_i.$$

After verifying the identity in the correct polynomial ring, evaluation at a common zero of the g_i proves $p = 0$ there. A certificate of p^k in the ideal is a different claim: concluding $p = 0$ can require a reducedness or field hypothesis. A certificate over $\mathbb{Q}[X]$ does not automatically prove the corresponding ideal membership over $\mathbb{Z}[X]$.

Arity checks precede pairing of coefficient vectors, atoms, generators, or certificate components. A truncated `zip` is not a proof that omitted components are irrelevant. Every bridge binds the complete original expression, not just its normalized printed spelling.

13.3 A shared reifier framework, not an untyped universal normalizer

Several checkers can share a typed reification framework and source-to-expression proof construction. They need not share one untyped arithmetic universe. Natural lengths, integer polynomials, rational affine forms, and real-valued inequalities have different operations and embeddings.

The reusable component should therefore be indexed by its algebraic signature, carrier, and named embeddings. It provides expression formation, atom identity, and compositional denotation lemmas. An affine normalizer, polynomial checker, and order certificate then supply different soundness theorems over appropriate instances of that framework.

For inequalities, coefficient signs and denominator directions matter. Mapping a natural length into $\mathbb{Q}$ can make affine comparison convenient, but returning a statement about natural lengths requires the corresponding injectivity or order-reflection theorem. The framework records that route rather than erasing the distinction between the source and target number systems.

13.4 Verified algorithms and checked certificates

Clover admits two principal routes. A verified algorithm computes its result inside a proved semantic framework. Alternatively, an untrusted producer returns a certificate checked by a

verified checker. Existing proof-producing tactics are another provider route whose generated proof term is checked in the usual way.

A theorem about a checker does not verify an arbitrary native execution of that checker. The strict route retains a term whose decisive computations are checked by kernel reduction or reconstructs a proof. Current Lean documentation records native evaluation through additional axiom dependencies, including per-invocation axioms for `native_decide`; Clover must audit the actual pinned toolchain rather than assume an older mechanism [15].

There is no measured certificate-size threshold in this article at which a native route becomes preferable. Such a threshold would depend on the checker, certificate distribution, hardware, proof policy, and retained evidence format. The default proof policy is chosen before such optimization.

13.5 Effectful programs are a separate contract axis

For stateful or effectful programs, pure input/output predicates are insufficient. Clover should use a specified Hoare-style semantics, with explicit state, exceptions, resources, and a distinction between partial and total correctness. The relation to weakest-precondition transformers is useful here; Dijkstra monads provide a relevant account of such specification structure [20].

Lean's `mvcgen` is an existing verification-condition-generation component, not a feature Clover needs to reinvent. The round-three reports' concrete Lean 4.32 probe described that interface as experimental; a production adapter must be pinned and tested [17, 2]. A synthesis timeout is an operational event, never a premise inside a Hoare theorem.

14 Should Lean's kernel type system change?

14.1 Yes to richer source typing; distinguish the kernel question

The user-facing proposal does extend the typing discipline: operations request proved properties, functions have behavioral interfaces, and unfinished terms carry dependent obligations. All three can be translated into existing Lean. This is the recommended first implementation, not an assertion that kernel research is uninteresting.

There are two different motivations for a kernel extension. One is to express properties that existing types cannot express. The properties in this article do not supply such a motivation: predicates, dependent functions, subtypes, and structures already express them. The other is to make common coercions and transport behavior definitionally simpler or operationally cheaper. That is a real but measurable research question.

14.2 A primitive proof-explicit refinement former

A candidate primitive could have the following rules for $A : \mathsf{Type}_u$ and $P : A \to \mathsf{Prop}$:

$$\frac{a : A \qquad p : P(a)}{\mathsf{pack}(a, p) : \mathsf{Ref}(A, P)}, \qquad \frac{r : \mathsf{Ref}(A, P)}{\mathsf{forget}(r) : A},$$

with

$$\mathsf{forget}(\mathsf{pack}(a, p)) \longrightarrow a.$$

A subsumption operation would still require evidence:

$$\mathsf{weaken} : (\forall x, P(x) \to Q(x)) \to \mathsf{Ref}(A, P) \to \mathsf{Ref}(A, Q).$$

Its straightforward translation is Lean's subtype representation. The primitive must not allow a bare $a : A$ to be regarded as refined without constructing a proof of $P(a)$.

Such a primitive could make a frontend implementation more uniform. It does not, merely by existing, eliminate proof obligations or improve runtime storage. Existing subtype representations already erase proposition-valued evidence in compiled code, and proof irrelevance removes distinctions between proofs of one proposition [9, 10]. A convincing performance case must identify an overhead that remains after an optimized ordinary-Lean encoding.

14.3 A contract arrow is a useful surface constructor

An explicit behavioral arrow could be written schematically as

$$A \xrightarrow{P;Q} B := \{f : A \to B \mid \forall x, P(x) \to Q(x, f(x))\}.$$

It has introduction from a function and law proof, projection to the function, and proof-backed variance and composition. Dependent outputs generalize B to $B(x)$. Relational laws can be bundled alongside this arrow rather than forced into a unary postcondition.

Again, the logical meaning is already representable. The useful source-level extension is that applications request and propagate these interfaces without requiring every local function variable to be rebound to a new wrapper type. A primitive kernel arrow would need a reason beyond prettier printing.

14.4 Definitional functoriality is a more specific experiment

A less superficial kernel experiment concerns coercions through data constructors. For an unknown list xs, equations such as

$$\mathsf{map}(\mathrm{id}, xs) = xs, \qquad \mathsf{map}(g, \mathsf{map}(f, xs)) = \mathsf{map}(g \circ f, xs)$$

are ordinary propositional laws, not generally obtained by reducing the unknown list's constructors. Repeated coercions through dependent containers can make such laws relevant to elaboration and proof size.

Laurent, Lennon-Bertrand, and Maillard study definitional functoriality and its relationship to coercive subtyping in a dependent type theory [21]. That work is a serious reference point, not a drop-in soundness result for every feature of Lean's kernel. Clover could investigate a restricted coercion language whose composition is normalized canonically, first by theorem-backed elaboration and only then, if justified, by new conversion rules.

The initial experiment should compare three implementations of exactly the same typed coercion workload: ordinary library rewriting, a source-level canonicalizer that emits equality proofs, and a proposed kernel conversion extension. This separates convenience from a genuine need to change the trusted checker.

14.5 What the metatheory would have to establish

A kernel extension needs precise formation, introduction, elimination, and reduction rules, together with subject reduction, compatibility with universes and proof irrelevance, a decidable checking procedure for its stated fragment, and a conservativity or translation argument. A conversion extension also needs a coherent treatment of competing reduction and coercion paths.

For example, two isomorphisms of a vector space with itself can transport a vector differently. Identity and negation on a nontrivial real vector space are not interchangeable merely because

both are isomorphisms. A coercive subtyping scheme that silently chooses between them changes data, not just proofs. Coherence must therefore concern the actual chosen maps, not a slogan that isomorphic objects are "the same".

Lean4Lean offers relevant work toward a verified account of Lean's checking infrastructure, but it should not be cited as an already established proof of an unrelated kernel extension [22]. The extension must be checked against the actual version and supported feature set.

14.6 Why semantic entailment should not enter conversion by default

A rule identifying refined types whenever their predicates are logically equivalent would make type conversion depend on theorem proving unless the predicate fragment were carefully restricted. The objection is not that a timeout becomes false; a sound implementation would return unknown. The problem is that the foundational checking boundary would acquire an additional, potentially expensive and incomplete logical search dependency.

Clover can obtain most of the intended convenience by inserting explicit proof-directed coercions in the elaborator. The kernel then checks the resulting term rather than re-running the search. A deliberately restricted, decidable predicate-conversion fragment remains a possible experiment, but its grammar, decision procedure, and interaction with dependent types must be part of the proposal, not left implicit.

The practical decision is therefore: implement the richer author-facing type system now over ordinary Lean; retain proof-explicit native refinements and restricted definitional coercions as testable research options; move no open-ended property inference into kernel conversion.

15 The executable companion: what was actually built

15.1 An intentionally small source language

The companion implements endomorphisms of one arbitrary carrier and four properties: injectivity, involution, left inverse, and pointwise equality. It has function binders, assumptions, claims, nested lexical scopes, and written composition expressions. Its actual syntax is:

```
fix f g r k : End
assume retraction : left_inverse r f
assume outer : injective g
claim composite : injective (g . f)
assume renamed : same g k
claim transported : injective (k . f)

scope
  fix j : End
  assume local : involutive j
  claim local_composite : injective (j . f)
end

claim unsupported_strengthening : involutive k
```

The first three claims obtain checked proof plans. The last remains unresolved: no rule or premise establishes that k is an involution. It is not marked false. A derived claim can supply evidence later; an unresolved claim cannot.

The symbol `.` in this restricted grammar means composition with the outer function first, so `(g .`

`f)` denotes $g \circ f$. All functions are endomorphisms of one carrier; dependent types, heterogeneous arrows, refinements of arbitrary Lean expressions, and the richer source notation in the article are not implemented by this parser.

15.2 Seven semantic rules and a separate plan checker

The fixed registry contains involution-to-left-inverse, left-inverse-to-injective, composition of injective maps, symmetry and transitivity of pointwise equality, transport of injectivity through pointwise equality, and reflexive pointwise equality. Their mathematical meanings are expressed in the ordinary-Lean file `CloverCore.lean`.

Search produces proof graphs. A separate checker reconstructs every rule application, checks the instantiated premises and conclusion, verifies that assumptions have the expected identities and propositions, checks scope, and compares the proof with the exact request. The checker shares the expression representation and substitution utilities with the generator; it is not an independently verified implementation of the semantics. It does not call Lean.

The exporter produces ordinary Lean declarations with shared intermediate `have` statements rather than recursively duplicating whole proofs. For the first example, the mathematical content of the generated body is:

```
have step1 : Injective f := inj_from_left_inverse f r retraction
have step2 : Injective (compose g f) := inj_comp f g step1 outer
exact step2
```

The actual generated identifiers include their unique local identities. The exported theorem retains all original ambient assumptions; its separate support report identifies the assumptions actually used. It does not silently publish a strengthened theorem by deleting unused premises.

15.3 Executed regression results

The 35 unit-test methods pass. They cover positive composition and transport, seedless cycles, missing premises, resource exhaustion, changed snapshot and target identities, forged substitutions, forbidden rules, a required root rule, sibling-scope isolation, shadowed names, pending claims, malformed syntax, and ordinary Lean export. One method compares opposite closure schedules across 32 seed subsets. Another exhausts 136 valuations of the seven rules over all functions on a two-element carrier.

Those finite semantic checks are not universal proofs of the rules. The ordinary mathematical proofs of the rules are simple, and their Lean reference contains no `sorry` or added axioms in its source, but it was not compiled in this environment. Neither the source inspection nor the Python checks are reported as kernel acceptance.

A separate exact-arithmetic program checks the affine test-basis construction, its concrete realizers, 625 small affine forms, finite grids for accepted forms, empty-element-type and guarded-length examples, and the Frey fixtures. Its report records 1,576 individual assertion checks. This count is not added to the number of unit-test methods as though they were the same unit of evidence.

15.4 A small but informative profile experiment

Metric, same first composition request	Seven rules	Two-rule profile
Arena terms	5	5
Candidate instantiations / retained ground rules	211	16
Relevant ground atoms	60	18
Rule tests in repeated-scan closure	633	48
New closure facts	7	2
Nodes in retained proof, including assumptions	4	4

The narrow profile permits only left-inverse-to-injective and injective composition. The reference implementation uses repeated scans, not the optimized premise-counter closure described earlier. Its counters therefore expose both grounding cost and avoidable closure work. The experiment supports a specific engineering decision—make profiles small and inspectable—not a claim that Clover is faster than Lean, `fun_prop`, or direct theorem lookup.

15.5 What these tests do not cover

The companion does not implement a real Lean local-context index, universe metavariables, instance selection, dependent transport, analytic locality, Frey property-implicit calls, full behavioral synthesis, a CAS certificate bridge, or incremental editor migration. It does not compile the generated Lean files. It does not run the full round-three negative suite, reproduce the syntheses' reported kernel experiments, or measure user comprehension.

Its advance is narrower: a real parser and executable finite grounding policy produce scope-sensitive proof plans and ordinary Lean source, and the test suite mutates the same artifacts at the correspondence boundaries. The next useful step is to port this pipeline to Lean and replace the symbolic plan-checking boundary with actual kernel-checked terms, not to enlarge the Python model until it becomes an unofficial second Lean.

16 A Lean implementation plan with explicit exit tests

16.1 Modules and responsibilities

The first Lean package should have a small separation of concerns. `Clover.Syntax` defines the structured source nodes and source spans. `Clover.Seal` classifies holes and retains the original typed targets. `Clover.Registry` validates theorem-role metadata. `Clover.Evidence` indexes local proofs and quantified schemes. `Clover.Obligations` stores typed pending requests and dependency edges. `Clover.Resolve` implements lookup, bounded grounding, and profile selection. `Clover.Export` reconstructs accepted terms and source-to-proof mappings. A renderer consumes these records rather than introducing another semantics.

Provider adapters are separate modules. They may call existing tactics or Leant, but they do not mutate the accepted evidence index directly. They return candidate results to a commit boundary that checks the exact target, dependencies, and policy before admitting any evidence.

16.2 Gate one: replace the symbolic boundary with Lean

Port the small endomorphism example first. The gate is not merely that the seven library theorems compile; it is that source claims elaborate into checked Lean terms and the same negative mutations fail at the intended boundary. A missing premise must leave an obligation.

A scope leak must be rejected. A forged result for another target must not be accepted because it is a true theorem.

The Lean implementation should log the actual theorem instances, number and size of retained expression nodes, grounding work, and kernel-checking outcome. The Python implementation can serve as a reference for the small finite grammar, but disagreements must be resolved semantically rather than by treating Python as an oracle for Lean.

16.3 Gate two: one property-implicit mathematical call

Implement `exact_quotient` and its transport interface over the existing Lean/Mathlib arithmetic lemmas. Reuse or extend the syntheses' already reported Frey artifacts rather than rebuilding a broad arithmetic library. Exercise the satisfiable fixture and the separated coefficient requirements from Proposition 12.1.

The exit test compares a property-implicit call with a normal Lean call supplied with the same helper theorems and automation. If the new interface adds unpredictable search while removing only a few argument names, the result may belong in a library or automatic-parameter helper rather than a new language.

16.4 Gate three: quantified locality

Implement one regional-equality scheme and the consumer turning it into a neighborhood equality at a member of an open set. Use the autonomous-derivative proof without strengthening its attained-range differentiability assumptions. The negative tests remove openness, specialize the induction hypothesis too early, or replace neighborhood equality by equality at one point.

This gate tests something the endomorphism model cannot: binder scope, quantified theorem reuse, theorem-role matching, and proof transport through actual analytic interfaces. Existing `fun_prop` rules and ordinary Lean wrappers are controls, not competitors to be excluded from the comparison [16].

16.5 Gate four: a behavioral denotation bridge

Port a small admitted list grammar and its length soundness proof into the Leant adapter. The required end-to-end result is a proof about the exact candidate's original contract. Then attach a realizable affine test basis, including the correlated-pair and empty-element cases.

The gate fails if the result is only a successful model check with no source bridge, if a passive provider assertion is treated as a Lean law, or if a counterexample lacks a source input satisfying the declared guard. Piecewise models need explicit case coverage. A broad solver connection is not a substitute for this narrow correspondence proof.

16.6 Gate five: replay, repair, and publication

Add snapshot-local invalidation and typed transport for one selected class of rewrites. Test changes of binder identity, structure dictionary, measure, universe instantiation, dependent index, and method policy. Initially rebuild rather than migrate whenever correspondence is uncertain.

Publication requires two separate checks. Every asserted document node must have an accepted proof or be visibly marked unfinished; and every exported mathematical result must meet its pinned axiom policy. A failed unused assertion blocks a fully verified document even when

another route proves the final theorem. An ordinary partial document may still be saved and inspected; it must not be mislabeled as completely checked.

17 Evaluation that can disconfirm the language proposal

17.1 Separate infrastructure from notation

Use five principal experimental arms. L_0 is stock Lean and the project's existing tools. L_1 adds the new mathematical packages and theorem registrations but uses ordinary Lean syntax. L_2 adds the same search, checking, and certificate services available to Clover. L_3 adds property-aware use sites and the shared obligation engine. L_4 adds the proposed mathematical surface. A renderer-only control isolates the benefit of a more readable view without a new input language.

Packages, wrapper lemmas, and relevant existing automation must be available consistently across the appropriate arms. A one-line Clover invocation is not compared with an unassisted Lean proof that has been denied the theorem it calls. The most informative comparisons are L_3 versus L_2, and L_4 versus L_3 plus the renderer control.

17.2 Measure semantic work, not just characters

Primary authoring measures should include time spent actively constructing and repairing proofs, interventions needed to supply missing meaning, and incorrect interpretations of compact statements. Machine measures should separate elaboration, grounding, external search, proof construction, kernel checking, proof-term size, and retained index memory. Report distributions and outliers, not only a mean.

A reading experiment should ask participants to reconstruct the domain, quantifier order, hypotheses, conclusion strength, and claimed proof method before seeing the expanded view. The almost-everywhere quantifier example, exact quotient cast, and finite tensor construction are suitable development items because their omitted distinctions can be stated precisely. They are not held-out items after they have influenced this design.

All examples reviewed in this article belong to the development set. Held-out tasks should be selected after the interfaces are fixed, include already-short Lean proofs, and include cases where the intended inference fragment fails. Participants and task order should be counterbalanced so that familiarity with one source does not masquerade as a language benefit.

17.3 Negative tests need a positive neighbor and a failure kind

Every semantic mutation should record a satisfiable positive fixture where applicable, the changed premise or target, and the expected failure category. Some representative cases are:

Positive situation	Mutation	Required outcome
A left inverse of the same function	Rename or replace that function at the consumer	Correspondence failure or unresolved property.
Regional equality near a point	Keep only equality at the point	Locality obligation remains.
An exact integer quotient	Remove divisibility, keeping the same rational-looking notation	Exactness obligation remains.

Positive situation	Mutation	Required outcome
A target-side unit	Keep only source-side nonzeroness	Target inverse obligation remains.
A finite family of finite free factors	Remove finite generation of one factor	Construction contract is not established.
Countably many a.e. equalities	Replace the family by an arbitrary uncountable one	Uniform-null-set obligation remains.
A concrete length counterexample	Use an unrealizable or unguarded observation	No source refutation.
A law about one candidate	Change its term graph or displayed edit	Recheck; old receipt is insufficient.
A completed proof document	Insert an unused unsupported assertion	Document is not fully checked.
A method-selected derivation	Supply a different proof route without the requested method	Method-policy failure, not mathematical falsity.

Some failures are logical counterexamples; others are deliberately outside an automatic fragment or violate correspondence policy. The test runner must not collapse them into a single boolean meaning "false." The companion executes only the subset described in Section 15; this table is the larger Lean implementation's acceptance specification.

17.4 Stopping rules

If the library and service arms achieve the same authoring improvement as the property layer, retain the library, providers, and renderer. If the property layer helps but the new syntax harms comprehension or repair, retain the property-aware Lean extension without a separate language. If a small profile works well but broad automatic inference is unpredictable, keep the small profile and explicit method selection.

These are not retreats from the project. They are the distinctions needed to identify which part of the design actually solves the observed problem. No human productivity result or Lean elaboration speedup is claimed in this article.

18 Answers to the round-three implementation questions

The syntheses' questions overlap. The following table gives a concrete disposition for the most important shared demands and the ten explicitly numbered questions in *Nine Refinements*, without pretending that a design answer is a completed experiment.

Question or demand	Clover's answer and current status
T1: cost on a real Lean file	Still unmeasured. The companion reports model operation counts; the implementation gates require separate real Lean timings, proof sizes, and index memory.
T2: which rules fire?	Retain exact proof-DAG rule instances and all grounding counters. The first model proof uses two rules; the full seven-rule profile generates substantial unused search. Real-file comparison with direct lookup remains open.

Question or demand	Clover's answer and current status
T3: rewritten anchors	Preserve the original anchor and use checked equality or proposition-equivalence bridges. Weaker relations require consumer theorems. General Lean rewrite migration is not implemented here.
T4: a common reifier	Share a typed signature-indexed framework and denotation construction, not one untyped arithmetic grammar. Original-target bridges remain checker-independent obligations.
T5: abstraction completeness	Theorem 11.2 supplies a complete affine criterion for an observation image with coverage and realizers. It does not establish a bridge for every current Leant Length feature.
T6: almost-everywhere rows	Store the measure and full binder order. Register integral consumers and countable-family collection explicitly; point evaluation is not a default consumer.
T7: lexical structure contexts	Fix actual dictionaries and create local instance bridges at use sites. No generated-preamble shortening experiment was performed.
T8: bounded rule generation	Section 9 specifies source-derived arenas and goal-triggered grounding; the companion implements this for a fixed endomorphism grammar.
T9: reader recovery	Use blind reconstruction of domain, quantifiers, hypotheses, conclusion strength, and method. No reading study has been conducted.
T10: independent implementations	The symbolic checker and search engine are separate paths but share representations and are not independent Lean implementations. A Lean port compared against the model is the next useful comparison.
Witness choice and method fidelity	Classify holes by semantic role; permit new data only at construction nodes. Check an explicit derivation skeleton or declared rule policy rather than a decorative method name.
Trusted execution and migration	Retain proof/certificate routes under a pinned axiom policy. Recheck after environment changes; no measured native-evaluation crossover is claimed.
First interface and exit condition	Start with the narrow checked pipeline, then exact quotients and regional differentiation. Keep only the library or renderer if controlled comparisons do not support the language layer.

19 Conclusion

Clover's central claim is not that mathematical rigor needs fewer obligations. It is that authors should be able to state the mathematical scope and transformation that generate those obligations, while a disciplined elaborator assembles the routine evidence.

The round-three refinement calculus is a suitable foundation. The next step is to make its operational boundary concrete: retain quantified evidence; represent unfinished results as genuinely conditional terms; restrict routine inference to explicit finite fragments; preserve object identity through rewriting and replay; and equip behavioral observations with the coverage and realization theorems their use requires.

The executable slice demonstrates part of that boundary and exposes a cost that a purely architectural proposal could hide. The affine test-basis theorem shows how richer behavioral typing can improve both positive verification and negative witness validity without extending kernel conversion. The source studies show where the same discipline matters in analysis,

arithmetic, and algebraic construction.

The recommended Clover implementation is therefore a small proof-producing extension of Lean, evaluated against equally equipped ordinary Lean, with a mathematical surface only where it earns its place. Native refinements or restricted definitional coercions remain precise research options, not prerequisites for beginning that implementation.

A Reproducing the companion

The package uses Python's standard library only. From the `companion` directory, run:

```
python -m unittest -v test_clover_slice
python clover_slice.py demo.clover --json demo-result.json \
  --lean GeneratedExamples.lean
python affine_basis.py
```

The first command runs the 35 unit-test methods. The second re-creates the three accepted proof plans, the unresolved claim, and the ordinary-Lean export. The third re-creates the exact-arithmetic report. The saved profile comparison is reproducible by the command in the README.

To check the Lean reference and generated examples in a suitable installed Lean environment, make the companion directory available on Lean's import path, compile `CloverCore.lean` to `CloverCore.olean`, and then check `GeneratedExamples.lean`. The README gives platform-specific shell commands. No successful execution of those commands is claimed by this package. The intended first target for integration is the campaign's Lean 4.32.0 baseline, followed by an explicitly pinned upgrade test.

The article is self-contained LaTeX and builds with:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error Clover.tex
```

No external bibliography processor, downloaded font, or generated table file is required.

B A minimal accepted-result record

The full implementation's proof-bearing result record should preserve at least the following information. This is a schema specification, not a claim that the symbolic companion implements every field.

```
AcceptedStep {
  sourceNode      : SourceSpanAndStableNodeIdentity
  originalTarget  : TypedLeanExpr
  context         : DependentLocalContextSnapshot
  structureArgs   : ExplicitStructureArguments
  construction    : OptionalAuthorizedWitnessRecord
  derivation      : TypedMethodOrRuleDAG
  evidence        : LeanProofOrCheckedCertificateBridge
  conclusion      : TypedLeanExpr
  support         : TransitiveAssumptionAndDeclarationDependencies
  policy          : RuleAndAxiomPolicy
  environment     : PinnedToolchainAndImportManifest
  replay          : CheckedCorrespondenceToDisplayedSourceEdit
}
```

Hashes can index and bind serialized records, but cannot replace typed correspondence checks. A record with an unfinished obligation is a pending record with a different status, not an `AcceptedStep` with an optimistic flag. An external producer may report a candidate record; only the checked commit boundary constructs the accepted form.

C Evidence inventory

Artifact	Evidence provided
`Clover.tex`, `Clover.pdf`	Design, written proofs, source analysis, explicit limits, and evaluation specification.
`SOURCE_MANIFEST.json`	Exact upstream repository references and scope of direct inspection.
`clover_slice.py`	Executable restricted parser, finite grounding, proof-plan checking, and Lean export.
`test_clover_slice.py`	35 passing unit-test methods; some contain multiple finite cases.
`CloverCore.lean`	Ordinary-Lean definitions and proofs for the seven rules; not compiled in this environment.
`GeneratedExamples.lean`	Three exported theorem bodies and a comment marking the unresolved claim; not compiled here.
`demo-result.json`	Exact model verdicts, assumption support, and counters; all explicitly say kernel checking was not performed.
`profile-comparison.json`	Reproducible operation-count comparison on the same model request.
`affine_basis.py`, `affine-result.json`	Exact arithmetic, realizers, and 1,576 finite checks; not a verified source-denotation bridge.
`EXECUTION_STATUS.json`, `test-output.txt`	Execution scope and saved test output.

References

[1] Brainstorm campaign, *From Properties to Proof Obligations*, revised round-three synthesis, commit `bf33339059950608 70f8585e297ffc16f3fe7e86`. Main LaTeX report inspected on 6 September 2026. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf33339059950608 70f8585e297ffc16f3fe7e86/docs/round-3/synthesis/unified-report.tex.

[2] Brainstorm campaign, *Nine Refinements*, revised round-three unified report, same Brainstorm commit. Main LaTeX report inspected on 6 September 2026. https://github.com/VladimirReshetnikov/Brainstorm/blob/bf33339059950608 70f8585e297ffc16f3fe7e86/docs/round-3/unified_report/unified_report.tex.

[3] Vladimir Reshetnikov et al., ProveIt, *AutonomousIteratedDeriv.lean*, commit `24ce8bd743eaab64a91ce90725ea00f498d319d2`. https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d3 19d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean.

[4] Anthropic FLT repository, *S_FreyPackage_freyCurveInt_map.lean*, commit `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de 0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_freyCurveInt_map.lean.

[5] Anthropic FLT repository, *S_Algebra_exists_algHom_equiv_pi.lean*, same FLT commit. https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de 0d8e75b9a3e9ef/P2M/Sol/S_Algebra_exists_algHom_equiv_pi.lean.

[6] Vladimir Reshetnikov et al., Leant, *src/Leant/Synth/Verification.hs*, commit `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7 b425eccb/src/Leant/Synth/Verification.hs.

[7] Leant, *src/Leant/Synth/Length/Contract.hs*, same directly inspected commit. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7 b425eccb/src/Leant/Synth/Length/Contract.hs.

[8] Leant, *src/Leant/Synth/Length/Adapter.hs*, same directly inspected commit. https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7 b425eccb/src/Leant/Synth/Length/Adapter.hs.

[9] Lean Language Reference, *Subtypes*, online documentation accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/.

[10] Lean Language Reference, *Propositions*, accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/The-Type-System/Propositions/.

[11] Lean Language Reference, *Quantifiers*, accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/Basic-Propositions/Quantifiers/.

[12] Lean Language Reference, *Function Application*, accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/Terms/Function-Application/.

[13] Lean API documentation, *Lean.Elab.SyntheticMVars*, accessed 6 September 2026. https://lean-lang.org/doc/api/Lean/Elab/SyntheticMVars.html.

[14] Lean Language Reference, *Elaborators*, accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/Notations-and-Macros/Elaborators/.

[15] Lean Language Reference, *Axioms*, especially the parametricity example, native-evaluation discussion, and dependency audit, accessed 6 September 2026. The `latest` documentation observed here contains links to the 4.34.0-rc2 reference; it is not used as a substitute for the campaign's 4.32.0 test pin. https://lean-lang.org/doc/reference/latest/Axioms/.

[16] Mathlib documentation, *Mathlib.Tactic.FunProp*, accessed 6 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html.

[17] Lean Language Reference, *The mvcgen tactic: Overview*, accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/Overview/.

[18] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala, *Liquid Types*, PLDI 2008. https://doi.org/10.1145/1375581.1375602.

[19] Ranjit Jhala and Niki Vazou, *Refinement Types: A Tutorial*, arXiv:2010.07763, 2020. https://arxiv.org/abs/2010.07763.

[20] Danel Ahman et al., *Dijkstra Monads for Free*, POPL 2017; arXiv:1608.06499. https://arxiv.org/abs/1608.06499.

[21] Théo Laurent, Meven Lennon-Bertrand, and Kenji Maillard, *Definitional Functoriality for Dependent (Sub)Types*, extended version, arXiv:2310.14929; ESOP 2024. https://arxiv.org/abs/2310.14929.

[22] Mario Carneiro, *Lean4Lean: Towards a Verified Typechecker for Lean, in Lean*, arXiv:2403.14064, 2024. https://arxiv.org/abs/2403.14064.